

nyaml: A Format Converter for NeXus Data Model.

Rubel¹, Lukas^{1,3}, Andrea¹, Florian¹, Markus¹, Sherjeel¹,
Christoph¹, Heiko B.², José Antonio¹, Claudia¹, and Sandor¹

¹ Physics Department and CSMB, Humboldt-Universität zu Berlin, Berlin, Germany ² Lehrstuhl für Angewandte Physik, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany ³ Department Heterogeneous Reactions, Max Planck Institute for Chemical Energy Conversion, Mülheim an der Ruhr, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

The NeXus scientific data format ([Könnecke et al., 2015](#)) is a widely adopted standard for organizing and sharing scientific data, particularly in the field of materials characterization. NeXus definitions specify the hierarchical structure and semantics of valid NeXus files and are written in XML using the NeXus Definition Language (NXDL), which itself is specified using XSD (XML Schema Definition).

nyaml is a Python-based tool with both a command-line interface and an importable API that facilitates the conversion between NXDL XML and a simplified YAML representation. YAML's indentation-based syntax enhances human readability and simplifies manual editing. By providing a reliable, lossless round-trip conversion between XML and YAML, nyaml enables developers to edit NeXus definitions more efficiently without sacrificing structural or semantic fidelity.

Statement of need

The NeXus data format standard, which was originally introduced for neutron, X-ray, and muon science, has in recent years seen a significant enhancement across diverse scientific domains. The growth of both the standard and the number of NeXus definition developers makes it all the more important to ensure that the development process is both user-friendly and resilient to errors. The existing representation of NeXus definitions in XML offers structural rigor through the NeXus Definition Language (NXDL) and a well-defined hierarchy for metadata and data types. However, it is verbose and can be difficult to edit by hand. Writing and maintaining NXDL files often involves dealing with deeply nested elements and strict syntax, which can be error-prone and time-consuming, especially for users who are not familiar with XML development. nyaml addresses these challenges by enabling the development of NeXus definitions in a cleaner, YAML-based format while preserving the full structure, semantics, and developer comments of the original XML.

nyaml Converter

The **nyaml** tool is a Python package, containing the several modules, that provides a command line interface for converting NeXus application definitions or base classes from YAML file format (**.yaml** extension) into the XML file format (**.nxdl.xml** extension) and vice-versa. To write the NeXus application definitions or base classes in YAML format, the **nyaml** package introduces a set of the keywords and syntactic rules (see full (Documentation)) that are specific to the NXDL (NeXus Definition Language) in the YAML format. The **nyaml** package

40 is designed to be used as a command line tool, but it can also be imported as a Python module
41 for programmatic use.

42 The converter from the command line is invoked by the `nyaml2nxdl` registered in the `nyaml.cli`
43 module. In that module, the function `launch_tool` decides upon the input file type which what
44 conversion (either from YAML to XML or XML to YAML) will be invoked.

45 **YAML to XML:** The `nyaml.nyaml2nxdl` contains the necessary functions, classes, and methods
46 to implement the algorithm for converting the NXDL schema from YAML to XML. The
47 function `nyaml2nxdl` is the main entry point for executing the conversion from YAML to XML
48 with the execution of three steps: 1. properly collects and tracks comments in YAML file, 2.
49 parses the YAML file into a Python dictionary object, and finally 3. writes the XML tree in a
50 file in accordance with the NXDL concepts including the comments from the first step. The
51 conversion algorithm uses the keywords and syntactic rules to read the NXDL in the YAML
52 format to transcode from the YAML to the XML representation. Taking leverage of the NeXus
53 NXDL rules, the conversion detect the possible inconveniences in the YAML file and raises an
54 error if the NXDL rules are not properly followed.

55 **XML to YAML:** The `nyaml.nxdl2yaml` module contains the necessary functions, classes, and
56 methods to implement the algorithm for converting the NXDL schema from XML to YAML.
57 The class `Nxdl2yaml` is the main entry point containing methods and global variables for
58 implementing the conversion algorithm from XML to YAML with the execution of three steps:
59 1. parsing the XML file into an XML tree object, 2. writing the YAML file in accordance
60 with the NXDL concepts, and 3. creates a hash from the YAML content using the SHA256
61 algorithm and extend the YAML file including the hash and original xml content as comment
62 respectively. Attaching the XML content with a hash at the end of the YAML file allows for a
63 lossless round-trip conversion if no modifications are made in the YAML content (as SHA256
64 creates the same hash from the same YAML content). Such caching mechanism offers clean
65 and efficient comparison of the XML content in version control systems like Git upon the
66 conversion from YAML to XML.

67 Evaluation from NAIC

68 The NeXus International Advisory Committee (NIAC) is the governing body responsible for over-
69 seeing the development and maintenance of the NeXus data standard. A core responsibility of
70 the NIAC is the stewardship of the NeXus Definition Language (NXDL), the XML-based schema
71 that defines the hierarchical structure and semantics of NeXus data files [Koennecke:2015].
72 As part of its mission to facilitate the standardization of NeXus definitions in NXDL, NIAC
73 has recently reviewed and formally accepted `nyaml`. Following a successful evaluation, NIAC
74 has approved `nyaml` and endorsed it as the recommended tool for preparing NeXus definition
75 proposals. In support of this decision, the official NeXus definition repository was updated to
76 integrate `nyaml` into its workflow through the addition of two makefile targets: `'make nyaml'`,
77 which converts existing definitions from the canonical `nxdl.xml` format into `.nyaml`, and `'make`
78 `nxdl'`, which detects modified or newly added `.nyaml` files and converts them back into valid
79 `nxdl.xml` format for submission and version control. This integration ensures that contributions
80 made in `.nyaml` are compatible with the existing XML-based infrastructure. The adoption
81 of `nyaml` by NIAC reflects an ongoing commitment to fostering community engagement and
82 modernising the technical tools underpinning the NeXus standard (Committee, 2025).

83 Figures

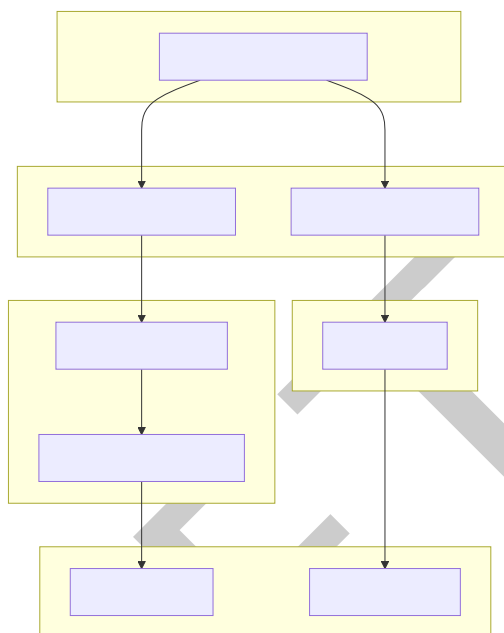


Figure 1: Caption for example figure.

84 Acknowledgements

85 References

- 86 Committee, N. I. A. (2025). *NIAC and NeXus documentation*. <https://www.nexusformat.org>
- 87 Könnecke, M., Akeroyd, F. A., Bernstein, H. J., Brewster, A. S., Campbell, S. I., Clausen, B.,
88 Cottrell, S., Hoffmann, J. U., Jemian, P. R., Männicke, D., & others. (2015). The NeXus
89 data format. *Journal of applied crystallography*. *Applied Crystallography*, 48(1), 301–305.
90 <https://doi.org/https://doi.org/10.1107/S1600576714027575>